

Apurba Koirala

22BCE3799

Deep Learning Assessment-4

Guided by: Dr. Sharmila C

Implement an RNN-based text classification model using the Twitter US Airline Sentiment dataset from TensorFlow Datasets. This dataset contains tweets about major U.S. airlines, labeled as positive, negative, or neutral. Perform the following steps:

- a Load and preprocess the data using TensorFlow's TextVectorization layer.
- b Build a Sequential model consisting of an Embedding layer, Bidirectional LSTM layers, and Dense layers for classification.
- c Train and validate the model, then plot training and validation accuracy and loss curves.
- d Test the trained model using custom tweet samples to predict their sentiment (positive, negative, or neutral)

```
! pip install kagglehub
```

```
Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (0.3.13)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (25.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2.3.0)
Requirement already satisfied: certifi<2024.12.1, >=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2024.12.14)
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("crowdfunder/twitter-airline-sentiment")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'twitter-airline-sentiment' dataset.
Path to dataset files: /kaggle/input/twitter-airline-sentiment
```

```
import os
```

```
path
```

```
'/kaggle/input/twitter-airline-sentiment'
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
```

```
df = pd.read_csv(path+'Tweets.csv')
```

```
df.head()
```

	tweet_id	airline_sentiment	airline_sentiment_confidence	negativereason	negativereason_confidence	air
0	570306133677760513	neutral	1.0000	NaN	NaN	Arr
1	570301130888122368	positive	0.3486	NaN	0.0000	Arr
2	570301083672813571	neutral	0.6837	NaN	NaN	Arr
3	570301031407624196	negative	1.0000	Bad Flight	0.7033	Arr
4	570300817074462722	negative	1.0000	Can't Tell	1.0000	Arr

Next steps:

[Generate code with df](#)[New interactive sheet](#)

df.isna().sum()

	0
tweet_id	0
airline_sentiment	0
airline_sentiment_confidence	0
negativereason	5462
negativereason_confidence	4118
airline	0
airline_sentiment_gold	14600
name	0
negativereason_gold	14608
retweet_count	0
text	0
tweet_coord	13621
tweet_created	0
tweet_location	4733
user_timezone	4820

dtype: int64

df.duplicated().sum()

np.int64(36)

df = df.drop_duplicates()

df.duplicated().sum()

np.int64(0)

df.shape

(14604, 15)

df = df.drop(['airline_sentiment_gold', 'negativereason_gold', 'tweet_coord'], axis = 1)

df.isna().sum()

	0
tweet_id	0
airline_sentiment	0
airline_sentiment_confidence	0
negativereason	5445
negativereason_confidence	4101
airline	0
name	0
retweet_count	0
text	0
tweet_created	0
tweet_location	4723
user_timezone	4814

dtype: int64

```
df = df.drop(['tweet_location'], axis = 1)
```

```
df = df.drop(['user_timezone'], axis = 1)
```

```
from sklearn.impute import KNNImputer
```

```
imputer = KNNImputer(n_neighbors=5, weights='distance')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 14604 entries, 0 to 14639
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tweet_id                             14604 non-null  int64
1   airline_sentiment                    14604 non-null  object
2   airline_sentiment_confidence         14604 non-null  float64
3   negativereason                       9159 non-null   object
4   negativereason_confidence            10503 non-null  float64
5   airline                              14604 non-null  object
6   name                                 14604 non-null  object
7   retweet_count                        14604 non-null  int64
8   text                                 14604 non-null  object
9   tweet_created                        14604 non-null  object
dtypes: float64(2), int64(2), object(6)
memory usage: 1.2+ MB
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["label"] = le.fit_transform(df["airline_sentiment"])
```

```
from sklearn.model_selection import train_test_split

# Assuming 'text' is the feature column and 'label' is the target column
X = df['text']
y = df['label']

# Split data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# Text preprocessing
max_features = 10000 # Define max_features
sequence_length = 128 # Define sequence_length

text_vectorization = tf.keras.layers.TextVectorization(
    max_tokens=max_features,
    output_sequence_length=sequence_length)

# Adapt the layer to the training texts
text_vectorization.adapt(X_train)

# Vectorize the text data
```

```
X_train_vec = text_vectorization(X_train)
X_val_vec = text_vectorization(X_val)
```

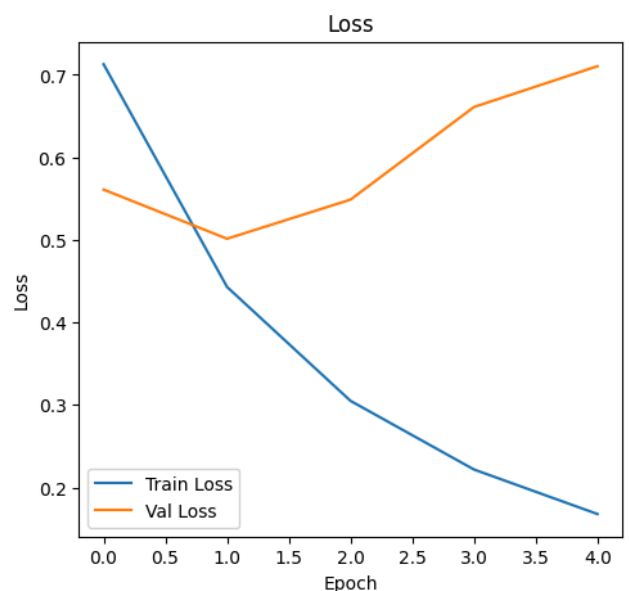
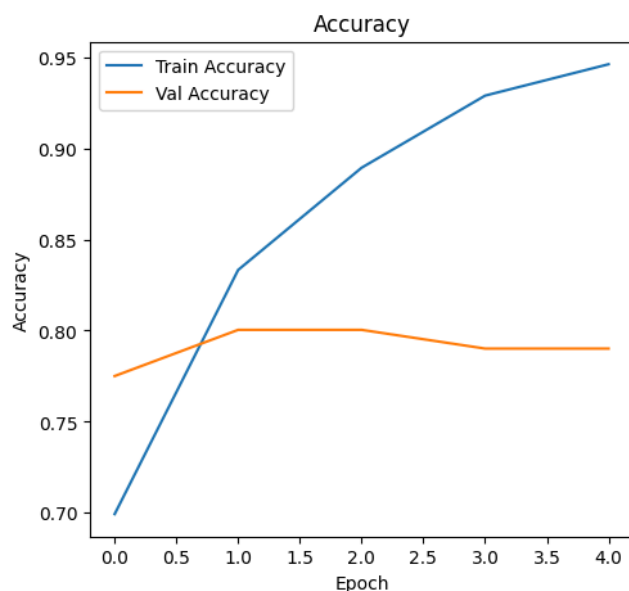
```
from tensorflow.keras import layers, models
from tensorflow.keras import regularizers
# Build RNN model (Bidirectional LSTM)
embedding_dim=128
model=models.Sequential([
    layers.Embedding(input_dim=max_vocab, output_dim=embedding_dim, input_length=max_len),
    layers.Bidirectional(layers.LSTM(64, return_sequences=True)),
    layers.Bidirectional(layers.LSTM(32)),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(3, activation='softmax')
])
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length`
warnings.warn(
```

```
history = model.fit(
    X_train_vec, y_train,
    validation_data=(X_val_vec, y_val),
    epochs=5,
    batch_size=64
)
```

```
Epoch 1/5
183/183 ————— 9s 27ms/step - accuracy: 0.6392 - loss: 0.8330 - val_accuracy: 0.7751 - val_loss: 0.5604
Epoch 2/5
183/183 ————— 5s 28ms/step - accuracy: 0.8301 - loss: 0.4472 - val_accuracy: 0.8004 - val_loss: 0.5071
Epoch 3/5
183/183 ————— 5s 25ms/step - accuracy: 0.8994 - loss: 0.2915 - val_accuracy: 0.8004 - val_loss: 0.5441
Epoch 4/5
183/183 ————— 5s 25ms/step - accuracy: 0.9307 - loss: 0.2207 - val_accuracy: 0.7901 - val_loss: 0.6601
Epoch 5/5
183/183 ————— 5s 27ms/step - accuracy: 0.9541 - loss: 0.1493 - val_accuracy: 0.7901 - val_loss: 0.7101
```

```
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```



```
loss, acc = model.evaluate(X_test_vec, y_test)
print(f"Test Accuracy: {acc:.4f}")
```

92/92 ————— 1s 10ms/step - accuracy: 0.7852 - loss: 0.7358
Test Accuracy: 0.7901

```
# Test with custom tweets
custom_tweets=[
    "I love the planes",
    "The flight was delayed again. Worst experience ever.",
    "It was okay"
]
custom_vec = vectorizer(custom_tweets)
pred = model.predict(custom_vec)
pred_labels = np.argmax(pred, axis=1)
label_decoder={0: 'negative', 1: 'neutral', 2: 'positive'}
for tweet, label in zip(custom_tweets, pred_labels):
    print(f"Tweet: {tweet}\nPredicted Sentiment: {label_decoder[label]}\n")
```

1/1 ————— 0s 54ms/step
Tweet: I love the planes
Predicted Sentiment: positive

Tweet: The flight was delayed again. Worst experience ever.
Predicted Sentiment: negative

Tweet: It was okay
Predicted Sentiment: neutral

Develop a transfer learning model using a pre-trained model such as ResNet50, InceptionV3, or MobileNetV2 to classify plant diseases from leaf images. First, train the model without fine-tuning and record its performance. Then, fine-tune the final layers, retrain the model, and compare the performance in terms of accuracy

```
from google.colab import files
files.upload()
```

kaggle.json
kaggle.json(application/json) - 69 bytes, last modified: 10/14/2025 - 100% done
Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username": "apurbakoirala", "key": "4dff4a2232d3a7ae133e8d859db1e4fd"}'}

```
# 2. Move it to the correct directory
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
# 3. Verify installation
!kaggle datasets list | head
```

ref	title
ahmadrazakashif/bmw-worldwide-sales-records-20102024	BMW Worldwide Sales Records (2010–2024)
jockeroika/life-style-data	Life Style Data
ahmeduzaki/global-earthquake-tsunami-risk-assessment-dataset	Global Earthquake–Tsunami Risk Assessment Dataset
jaderz/hospital-beds-management	Hospital Beds Management
grandmaster07/student-exam-score-dataset-analysis	Student exam score dataset analysis
anassarfraz13/coffee-sales-dataset	Coffee Sales Dataset
raghavdharwal/amazon-ml-challenge-2025	Amazon ML Challenge 2025
anassarfraz13/student-success-factors-and-insights	Student Success: Factors & Insights

```
!kaggle datasets download -d emmarex/plantdisease
!unzip plantdisease.zip -d plant_disease_data
```

[Show hidden output](#)

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import InceptionV3
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.optimizers import Adam
import matplotlib.pyplot as plt
import os
```

```
batch_size = 64
image_size = (224, 224)
train_dir = "plant_disease_data/PlantVillage"
```

```
datagen = ImageDataGenerator(rescale = 1/255, validation_split = 0.2)
```

```
train_generator = datagen.flow_from_directory(
    train_dir,
    target_size=image_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset = 'training'
)
```

Found 16516 images belonging to 15 classes.

```
validation_generator = datagen.flow_from_directory(
    train_dir,
    target_size=image_size,
    batch_size=batch_size,
    class_mode='categorical',
    subset = 'validation'
)
```

Found 4122 images belonging to 15 classes.

```
num_classes = len(train_generator.class_indices)
```

```
base_model = InceptionV3(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
```

```
base_model.trainable = False
```

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(1024, activation='relu')(x)
predictions = Dense(num_classes, activation='softmax')(x)
```

```
model = Model(inputs=base_model.input, outputs=predictions)
```

```
model.compile(optimizer = Adam(learning_rate=0.001),
              loss = 'categorical_crossentropy',
              metrics = ['accuracy'])
```

```
history = model.fit(
    train_generator,
    epochs=5,
    validation_data=validation_generator
)
```

```
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: You
self._warn_if_super_not_called()
259/259 ————— 88s 267ms/step - accuracy: 0.5994 - loss: 1.3834 - val_accuracy: 0.8132 - val_loss: 0.4
Epoch 2/5
259/259 ————— 51s 195ms/step - accuracy: 0.8324 - loss: 0.5026 - val_accuracy: 0.8370 - val_loss: 0.4
Epoch 3/5
259/259 ————— 52s 199ms/step - accuracy: 0.8640 - loss: 0.4039 - val_accuracy: 0.8343 - val_loss: 0.4
Epoch 4/5
259/259 ————— 50s 192ms/step - accuracy: 0.8834 - loss: 0.3434 - val_accuracy: 0.8435 - val_loss: 0.4
Epoch 5/5
259/259 ————— 46s 176ms/step - accuracy: 0.9045 - loss: 0.2774 - val_accuracy: 0.8540 - val_loss: 0.4
```

```
#finetuning
base_model.trainable = True
for layer in base_model.layers[:-50]:
    layer.trainable = False
```

```
model.compile(optimizer = Adam(learning_rate=0.0001),
              loss = 'categorical_crossentropy',
              metrics = ['accuracy'])
```

```
history_ft = model.fit(
    train_generator,
    epochs=5,
```

```
validation_data=validation_generator
```

```
\
```

```
Epoch 1/5
259/259 ————— 90s 264ms/step - accuracy: 0.8685 - loss: 0.3870 - val_accuracy: 0.9199 - val_loss: 0.2
Epoch 2/5
259/259 ————— 47s 182ms/step - accuracy: 0.9897 - loss: 0.0425 - val_accuracy: 0.9258 - val_loss: 0.2
Epoch 3/5
259/259 ————— 47s 181ms/step - accuracy: 0.9936 - loss: 0.0247 - val_accuracy: 0.9401 - val_loss: 0.1
Epoch 4/5
259/259 ————— 47s 182ms/step - accuracy: 0.9981 - loss: 0.0096 - val_accuracy: 0.9347 - val_loss: 0.2
Epoch 5/5
259/259 ————— 47s 179ms/step - accuracy: 0.9972 - loss: 0.0117 - val_accuracy: 0.9401 - val_loss: 0.2
```

```
loss_ft, acc_ft = model.evaluate(validation_generator)
print(f"Validation Accuracy with Fine-Tuning: {acc_ft:.4f}")
```

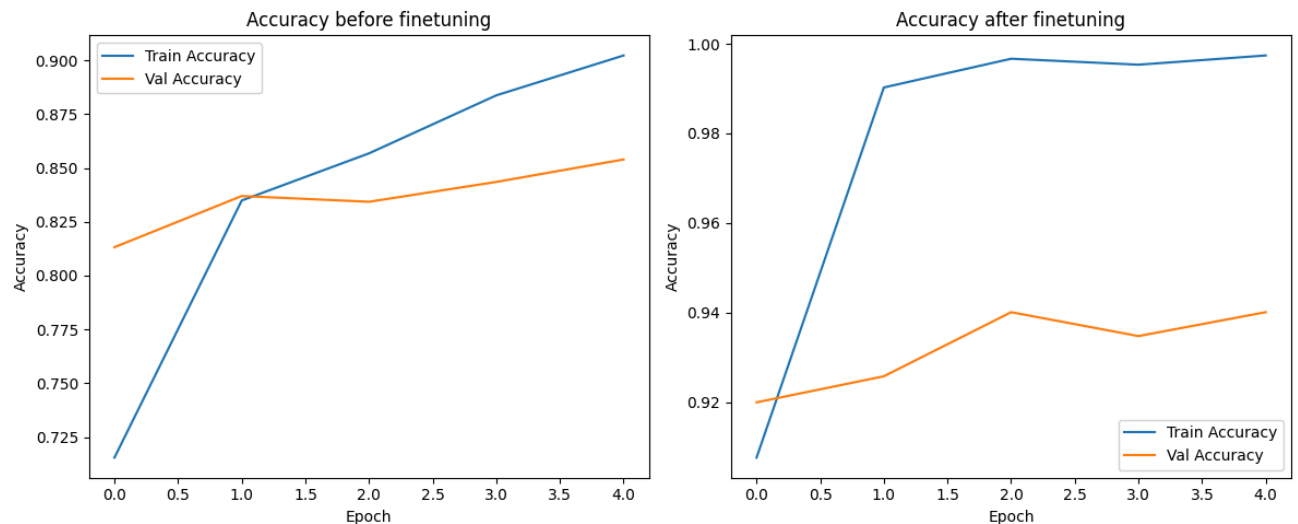
```
65/65 ————— 9s 138ms/step - accuracy: 0.9427 - loss: 0.2200
Validation Accuracy with Fine-Tuning: 0.9401
```

```
plt.figure(figsize=(12,5))
```

```
# Before fine-tuning
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy before finetuning')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
# After fine-tuning
plt.subplot(1,2,2)
plt.plot(history_ft.history['accuracy'], label='Train Accuracy')
plt.plot(history_ft.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy after finetuning')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.tight_layout() # <-- prevents overlap/clipping
plt.show()
```



Start coding or [generate](#) with AI.